

Laboratorio N.º1 — Alertas sonoras para el nodo de telemetría

1. Página de presentación

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Facultad de Ingeniería de Sistemas Computacionales

Departamento de Programación de Computadoras

Experiencia	Laboratorio N.º1: Alertas sonoras para el nodo de telemetría
Asignatura	Desarrollo de Software VIII
Docente	Kexy Rodríguez
Grupo	IGS134
Estudiante	Leonardo Castro 8-1032-1264
Fecha de entrega	01/10/2026
Repositorio	https://github.com/AC061/pythonLab1

2. Índice

1. [Página de presentación](#)
2. [Índice](#)
3. [Introducción](#)
4. [Objetivo](#)
5. [Descripción del sistema base](#)
6. [Mejora: conexión de red y alertas sonoras](#)
7. [Tabla de alertas sonoras y justificación](#)
8. [Cómo se evitó que el sonido bloqueara el ciclo de monitoreo](#)
9. [Capturas de pantalla de la ejecución](#)
10. [Resultado de las pruebas unitarias](#)
11. [Cumplimiento de los resultados esperados](#)
12. [Conclusiones](#)
13. [Uso de herramientas de inteligencia artificial](#)
14. [Anexo: instalación y ejecución](#)

3. Introducción

En el laboratorio de cómputo de la FISC, el técnico encargado deja el nodo de telemetría funcionando en una computadora mientras atiende otras tareas. Si el procesador se satura, la memoria se agota o se cae la red cuando el técnico no mira la pantalla, nadie se entera a tiempo, porque el dashboard solo avisa de forma visual.

El proyecto parte de un **monitor de sistema con interfaz gráfica**, con un comportamiento similar a **htop** pero más intuitivo: una **tarjeta por componente** (CPU, GPU, Disco y Red) que muestra su **uso, temperatura y capacidad**. En la CPU se muestran además los núcleos y en la GPU la memoria de video (VRAM). Está construido con **Python, customtkinter** (interfaz) y **HardwareMonitor** (sensores en Windows), con **psutil** como respaldo multiplataforma.

El laboratorio amplía ese nodo con dos capacidades:

1. **Detección de la conexión de red:** eventos por flanco cuando se pierde y se recupera la conexión, y un recordatorio por tiempo mientras siga caída.
2. **Alertas sonoras** con un sonido distinto para la CPU, la memoria, el tráfico de red y la conexión, que suenan **sin congelar** el dashboard ni el ciclo de monitoreo.

Gracias al diseño orientado a eventos del nodo (sensores → detectores → bus de eventos), el sonido se añadió como **una reacción más a los eventos**, sin modificar la lógica de los sensores ni de los detectores existentes.

4. Objetivo

Objetivo general. Ampliar el nodo de telemetría con alertas sonoras y detección de la conexión de red, manteniendo el ciclo de monitoreo y el dashboard siempre activos.

Objetivos específicos (los del enunciado):

1. Ampliar el nodo con una nueva reacción a los eventos, sin modificar la lógica de los sensores ni de los detectores.
2. Detectar la pérdida y la recuperación de la conexión de red mediante un evento por flanco y un recordatorio por tiempo.
3. Implementar un reproductor de alertas sonoras que no bloquee el ciclo de monitoreo, usando una cola (**deque**) y marcas de tiempo.
4. Verificar el funcionamiento mediante pruebas unitarias y pruebas en vivo.

5. Descripción del sistema base

5.1 Estructura de archivos

Todos los módulos están en la raíz del proyecto, más un `main.py`. La única carpeta es el paquete `alertas/`, que exige el enunciado (resultado E.1). Los archivos marcados con **(L)** son de la mejora del laboratorio.

Archivo	Responsabilidad
<code>main.py</code>	Punto de entrada. Opciones <code>--consola</code> y <code>--silencioso</code> .
<code>config.py</code>	Único lugar con números, umbrales, tiempos, sonidos, colores y nombres de sensores.
<code>modelos.py</code>	<code>dataclass</code> de las lecturas y de la <code>Instantanea</code> .
<code>hardware.py</code>	Acceso a HardwareMonitor (<code>ProveedorLHM</code>) y temperaturas con <code>psutil</code> .
<code>cpu.py</code> , <code>memoria.py</code> , <code>disco.py</code> , <code>red.py</code> , <code>gpu.py</code>	Sensores. <code>red.py</code> solo mide tráfico (KB/s).
<code>conexion.py (L)</code>	Sensor que indica si el equipo está conectado a una red.
<code>eventos.py</code>	<code>Evento</code> y <code>BusEventos</code> (publicar/suscribir).
<code>detectores.py</code>	Detectores por umbral con histéresis y, (L) , <code>DetectorConexion</code> .
<code>nodo.py</code>	<code>NodoTelemetria</code> : ciclo de monitoreo en un hilo propio.
<code>alertas/ (L)</code>	Paquete de alertas: <code>sonidos.py</code> , <code>backends.py</code> , <code>reproductor.py</code> .
<code>formatos.py</code> , <code>componentes.py</code> , <code>tarjetas.py</code> , <code>dashboard.py</code>	Interfaz con <code>customtkinter</code> .
<code>consola.py</code>	Modo texto, sin interfaz gráfica.
<code>demo_alertas.py (L)</code>	Prueba en vivo de las alertas sonoras.
<code>test_detectores.py</code> , <code>test_nodo.py</code>	Pruebas unitarias del sistema base.
<code>test_alertas.py (L)</code> , <code>test_conexion.py (L)</code>	Pruebas unitarias de la mejora.

5.2 Arquitectura y flujo de datos

HARDWARE (HardwareMonitor / psutil / sysfs)

|

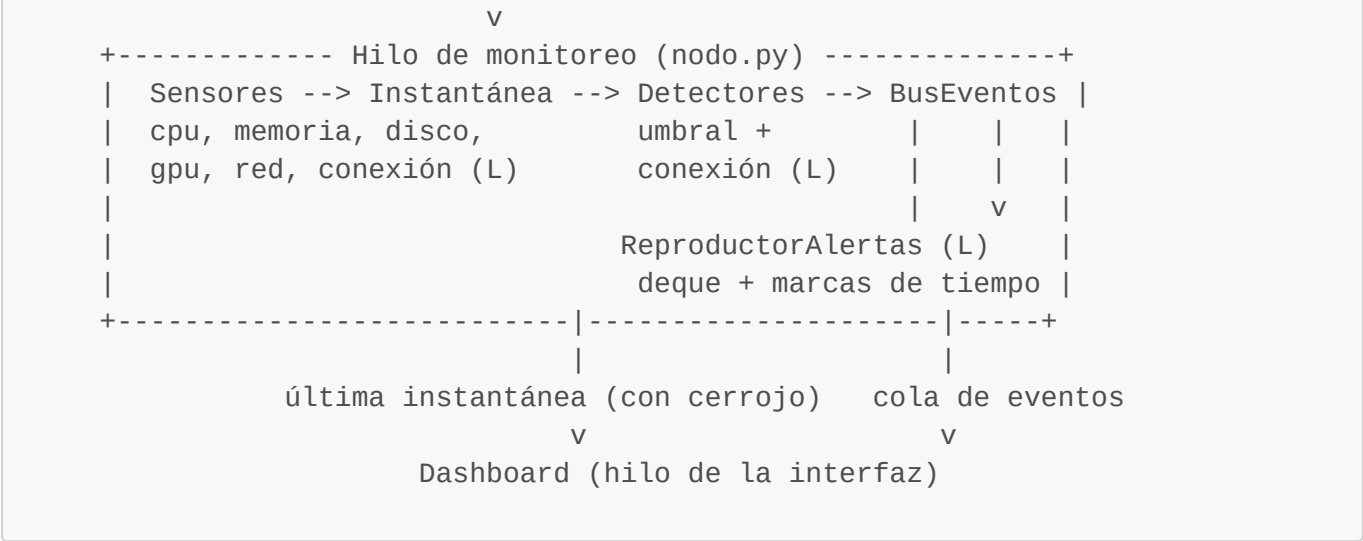


Figura 1. Arquitectura del nodo. El reproductor de alertas (L) es un suscriptor más del bus de eventos.

- **Sensores:** cada uno expone `leer()` y devuelve una lectura tipada. No conocen a los detectores ni a la interfaz.
- **Detectores:** reciben la `Instantánea` y emiten un `Evento` solo al cruzar un umbral (flanco). La zona entre el umbral *alto* y el *normal* es la histéresis, que evita repeticiones.
- **Bus de eventos:** reparte cada evento a los suscriptores de su nombre y a los comodín (*). Un suscriptor defectuoso no afecta a los demás.
- **Dashboard:** nunca toca el hardware; cada 250 ms consulta la última instantánea y vacía una cola de eventos.

5.3 Fuentes de datos por plataforma

HardwareMonitor envuelve LibreHardwareMonitor mediante `pythonnet` y **solo funciona en Windows**. El proyecto lo usa cuando está disponible y, si no, recurre a `psutil`, `sysfs` y `nvidia-smi`.

Dato	Windows (HardwareMonitor)	Linux / respaldo
Uso de CPU y núcleos	<code>psutil</code>	<code>psutil</code>
Temperatura de CPU	<code>Core (Tctl/Tdie)</code> , <code>CPU Package</code> , ...	<code>psutil.sensors_temperatures()</code>
GPU: uso, temperatura, VRAM	<code>GPU Core</code> , <code>GPU Memory Total/Used</code>	<code>nvidia-smi</code> o <code>sysfs /sys/class/drm</code> (AMD)
Disco: temperatura, actividad, E/S	Sensores de <code>Storage</code>	<code>psutil</code> y chip <code>nvme</code>
Red: tráfico	<code>psutil</code>	<code>psutil</code>

Nota de verificación. La ruta de HardwareMonitor no pudo probarse en el equipo de desarrollo (Linux). Se verificó con datos reales la ruta de respaldo (CPU AMD Ryzen 7 3700U, GPU AMD Vega, NVMe y WiFi). Debe probarse en Windows antes de la demostración.

5.4 Contenido de cada tarjeta

Tarjeta	Uso	Temperatura	Capacidad
CPU	% de uso total, una mini barra por núcleo y la RAM	°C	Núcleos físicos, lógicos y frecuencia
GPU	% de uso	°C	VRAM total y barra de VRAM usada
Disco	% de espacio usado, % de actividad, lectura/escritura	°C	Tamaño total (usado y libre)
Red	% de uso del enlace, bajada y subida	°C (si el SO la expone; si no, N/D)	Velocidad del enlace en Mbps

Las barras cambian de color según el nivel: verde, amarillo y rojo. Si un dato no está disponible se muestra N/D.

5.5 Eventos del sistema base

Los detectores se definen como datos en `config.DETECTORES`. Cada uno emite dos eventos por flanco.

Evento al superar	Evento al recuperarse	Umbral alto	Umbral normal
<code>cpu_alto</code>	<code>cpu_normal</code>	90 %	75 %
<code>cpu_temp_alta</code>	<code>cpu_temp_normal</code>	85 °C	75 °C
<code>memoria_alta</code>	<code>memoria_normal</code>	90 %	80 %
<code>gpu_alto</code>	<code>gpu_normal</code>	95 %	80 %
<code>gpu_temp_alta</code>	<code>gpu_temp_normal</code>	85 °C	75 °C
<code>disco_lleno</code>	<code>disco_espacio_normal</code>	90 %	85 %
<code>disco_temp_alta</code>	<code>disco_temp_normal</code>	70 °C	60 °C
<code>red_trafico_alto</code>	<code>red_trafico_normal</code>	10 240 KB/s	5 120 KB/s

5.6 Regla «sin números literales fuera de `config.py`»

Todos los umbrales, tiempos, frecuencias, duraciones, volúmenes, tamaños de fuente, márgenes y factores de conversión están en `config.py`. Solo se aceptan como literales los valores estructurales `0`, `1` y `-1` (índices y comparaciones) y los índices de fila/columna de `grid()`. Los archivos de pruebas usan valores literales en sus datos de ejemplo.

6. Mejora: conexión de red y alertas sonoras

6.1 Cambios realizados y marcado # LABORATORIO

Cada línea o bloque agregado o modificado lleva el comentario # **LABORATORIO**; los archivos nuevos lo llevan en su encabezado y en cada bloque.

Archivo	Tipo	Qué se agregó o cambió
<code>alertas/ (__init__, sonidos, backends, reproductor)</code>	Nuevo	Paquete de alertas sonoras
<code>conexion.py</code>	Nuevo	<code>SensorConexion</code>
<code>demo_alertas.py</code>	Nuevo	Prueba en vivo
<code>test_alertas.py,</code> <code>test_conexion.py</code>	Nuevo	45 pruebas unitarias
<code>detectores.py</code>	Ampliado	Clase <code>DetectorConexion</code> al final; <code>DetectorUmbral</code> no se tocó
<code>modelos.py</code>	Ampliado	<code>LecturaConexion</code> y el campo <code>conexion</code> en <code>Instantanea</code>
<code>config.py</code>	Ampliado	Sonidos, tiempos, eventos de conexión, mensajes y textos de la interfaz
<code>nodo.py</code>	Modificado	Registra el sensor y el detector de conexión; llama a <code>reproductor.actualizar()</code> cada ciclo
<code>main.py, consola.py</code>	Modificado	Opción <code>--silencioso</code> y creación del reproductor
<code>dashboard.py</code>	Modificado	Indicador de conexión, interruptor de sonido y botón de prueba

Sin modificar: los sensores `cpu.py`, `memoria.py`, `disco.py`, `red.py`, `gpu.py`, y la lógica de `DetectorUmbral`, tal como pide el objetivo 1.

6.2 Detección de la conexión de red

`red.py` mide el tráfico, pero no indica si el equipo está conectado, por eso no se tocó. La conexión se detecta con un sensor y un detector nuevos.

Sensor `SensorConexion` (`conexion.py`). Considera *conectado* al equipo si existe al menos una interfaz activa (que no sea loopback ni virtual: `lo`, `docker`, `veth`, `virbr`, ...) con una dirección IPv4 o IPv6 que no sea de loopback, de enlace local ni sin especificar. Es una comprobación local: no envía tráfico, por lo que no puede bloquear el ciclo. Limitación: no garantiza que haya salida a Internet.

Detector `DetectorConexion` (`detectores.py`). Es una máquina de tres estados (sin observaciones, conectada, desconectada) que emite:

Evento	Tipo	Cuándo se emite	Severidad
<code>red_desconectada</code>	Flanco	La conexión pasa de conectada a desconectada (o el nodo arranca sin red)	alerta
<code>red_conectada</code>	Flanco	La conexión pasa de desconectada a conectada; informa los segundos que estuvo caída	info
<code>red_sigue_desconectada</code>	Recordatorio por tiempo	Mientras siga caída, cada <code>RED_RECORDATORIO_S</code> (30 s)	alerta

```
(inicio) --conectada-----> CONECTADA
(inicio) --desconectada--> [red_desconectada] --> DESCONECTADA <--+
CONECTADA --desconectada--> [red_desconectada] --> DESCONECTADA |
DESCONECTADA --pasan 30 s--> [red_sigue_desconectada] -----+
DESCONECTADA --conectada--> [red_conectada] -----> CONECTADA
```

Figura 2. Estados del detector de conexión y eventos que emite.

El tiempo del recordatorio se calcula con la marca de tiempo de la instantánea; el detector no usa `sleep` ni temporizadores, así que nunca bloquea y es fácil de probar.

6.3 El paquete `alertas`

Módulo	Responsabilidad
<code>sonidos.py</code>	Convierte cada patrón de <code>config.ALERTAS_SONORAS</code> en un WAV en memoria (onda senoidal de 16 bits, con rampa de entrada y salida para evitar «clics»). Solo usa la biblioteca estándar.
<code>backends.py</code>	Formas de emitir el sonido, todas asíncronas : <code>winsound.PlaySound(..., SND_ASYNC)</code> en Windows; <code>Popen</code> de <code>pw-play</code> , <code>paplay</code> , <code>aplay</code> o <code>afplay</code> en Linux/macOS; campana del sistema como último recurso; y un backend silencioso.
<code>reproductor.py</code>	<code>ReproductorAlertas</code> : cola <code>deque</code> con marcas de tiempo y la función <code>crear_reproductor()</code> .

El reproductor se suscribe al bus para cada evento que tenga un sonido en `config.ALERTAS_SONORAS`. Al llegar un evento solo lo **encola**; el sonido lo inicia `actualizar()`, que se invoca en cada vuelta del ciclo de monitoreo.

6.4 Modo silencioso

Durante el trabajo en el salón se usa el modo silencioso, que no emite sonido pero conserva el registro: cada alerta aparece en el panel de eventos como `cpu_alto (silenciada)`.

- Al iniciar: `python main.py --silencioso` (o `ALERTAS_SILENCIOSO_POR_DEFECTO = True` en `config.py`).

- En ejecución: interruptor **Alertas sonoras** del dashboard.
- Sin sonido, la cola se consume de inmediato porque no hay nada que esperar.

7. Tabla de alertas sonoras y justificación

Los sonidos se definen en `config.ALERTAS_SONORAS` como tuplas (`frecuencia_hz`, `duracion_ms`); una frecuencia de 0 es una pausa. Todas las frecuencias están entre 300 y 1000 Hz, un rango que reproducen bien los parlantes pequeños de una computadora personal. Cada alerta se distingue por su **contorno** (sube, baja o se mantiene) y su **ritmo**, de modo que el técnico las reconozca de oído sin mirar la pantalla.

Evento	Patrón (Hz / ms)	Duración	Justificación
<code>cpu_alto</code>	600/110 → 800/110 → 1000/110	0.33 s	Tres tonos ascendentes, «acelerando». El tono que sube evoca un procesador subiendo de régimen. Es corto y agudo porque una saturación de CPU suele ser breve y recurrente.
<code>memoria_alta</code>	300/350 → pausa 100 → 300/350	0.80 s	Dos pulsos graves y largos. El tono grave y pesado sugiere algo «lleno» y es el más lento del conjunto, lo que lo separa claramente de los sonidos de CPU y de tráfico.
<code>red_trafico_alto</code>	(900/70 → 700/70) × 3	0.42 s	Ráfaga rápida de dos tonos alternados. El ritmo veloz imita paquetes de datos. Es el único sonido con alternancia, así que no se confunde con los de contorno único.
<code>red_desconectada</code>	900/200 → 600/200 → 300/350	0.75 s	Tres tonos descendentes. La caída de tono transmite «se perdió algo». Es el sonido más grave al final y el más largo de la red, porque es la falla más importante.
<code>red_conectada</code>	500/180 → 900/320	0.50 s	Dos tonos ascendentes. Tiene el contorno contrario al de la caída y es más breve y positivo: el técnico entiende que se recuperó sin mirar.
<code>red_sigue_desconectada</code>	400/150 → pausa 120 → 400/150	0.42 s	Dos pitidos graves y secos. Es un recordatorio que se repite cada 30 s, por eso es discreto: más corto que la alerta de caída y sin contorno, para no molestar si la falla se prolonga.

Los eventos de recuperación de CPU, memoria y tráfico (`cpu_normal`, `memoria_normal`, `red_trafico_normal`) no suenan: el enunciado pide distinguir *problemas*, y un sonido por cada recuperación saturaría el oído. Para asignar un sonido a cualquier otro evento basta con agregar una línea en `config.ALERTAS_SONORAS`.

Para evitar que alertas seguidas se solapen, entre dos alertas hay una pausa de `ALERTAS_PAUSA_ENTRE_S` (0.4 s).

8. Cómo se evitó que el sonido bloqueara el ciclo de monitoreo

El problema de fondo es que un sonido *dura*. Si el programa esperara a que termine (por ejemplo con `winsound.Beep`, que bloquea durante todo el tono, o con `time.sleep` entre tonos), el ciclo de monitoreo y el dashboard quedarían congelados. Se evitó con cuatro mecanismos:

- 1. **Emisión asíncrona.** Ningún backend espera a que termine el sonido. En Windows se usa `PlaySound` con `SND_ASYNC` desde memoria; en Linux y macOS se lanza el reproductor con `subprocess.Popen`, que devuelve de inmediato; la campana del sistema es una escritura instantánea. No se usa `winsound.Beep`.
- 2. **Cola deque con marcas de tiempo, sin esperas.** Al llegar un evento, el suscriptor solo agrega (`nombre`, `marca_de_tiempo`) al deque, una operación inmediata. En cada vuelta del ciclo, `actualizar()` compara el reloj con la marca de tiempo en que **terminará** la alerta en curso (`_fin_actual`): si aún no llega, devuelve de inmediato; si ya llegó, inicia la siguiente de forma asíncrona y calcula su nueva marca de fin (`ahora + duración + pausa`). El «temporizador» es una comparación de marcas de tiempo, no una espera.
- 3. **Protección de la cola.** El deque tiene tamaño máximo (`ALERTAS_COLA_MAX`, descarta la más antigua si se llena), no admite alertas repetidas pendientes y descarta las que esperaron más de `ALERTAS_CADUCIDAD_S` (20 s), porque un aviso viejo ya no describe el estado actual. Así una avalancha de eventos no se acumula sin límite.
- 4. **Hilos separados.** El monitoreo corre en su propio hilo y la interfaz nunca lee hardware: solo consulta la última instantánea y una cola de eventos con `after()`. Los suscriptores del bus están aislados, de modo que un error en uno no detiene el ciclo.

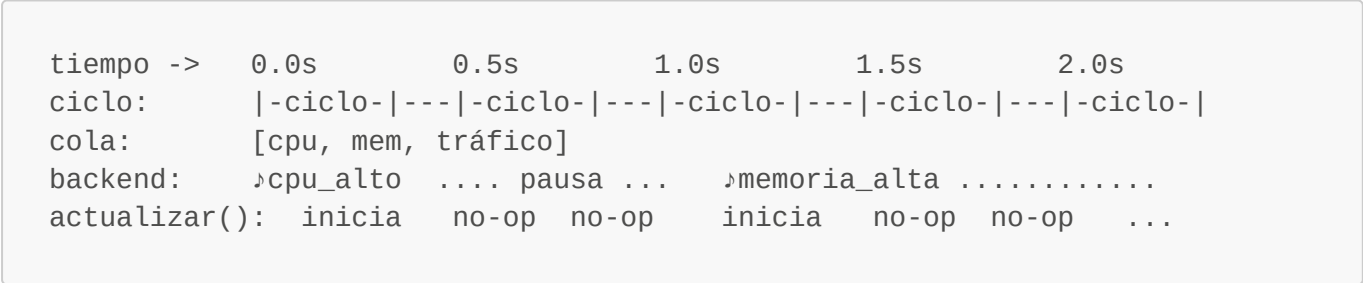


Figura 3. El ciclo continúa mientras suena cada alerta; `actualizar()` solo inicia la siguiente cuando vence la marca de tiempo.

Evidencia en vivo. `demo_alertas.py` simula un ciclo que «late» cada 0.25 s y encola de golpe las seis alertas. Con sonido real (`pw-play`), el ciclo imprimió **25 latidos seguidos sin ninguna interrupción** mientras sonaban las seis alertas, y la mayor duración de `actualizar()` fue de **1.92 ms**. Cada alerta comenzó en el primer latido posterior al fin de la anterior más la pausa:

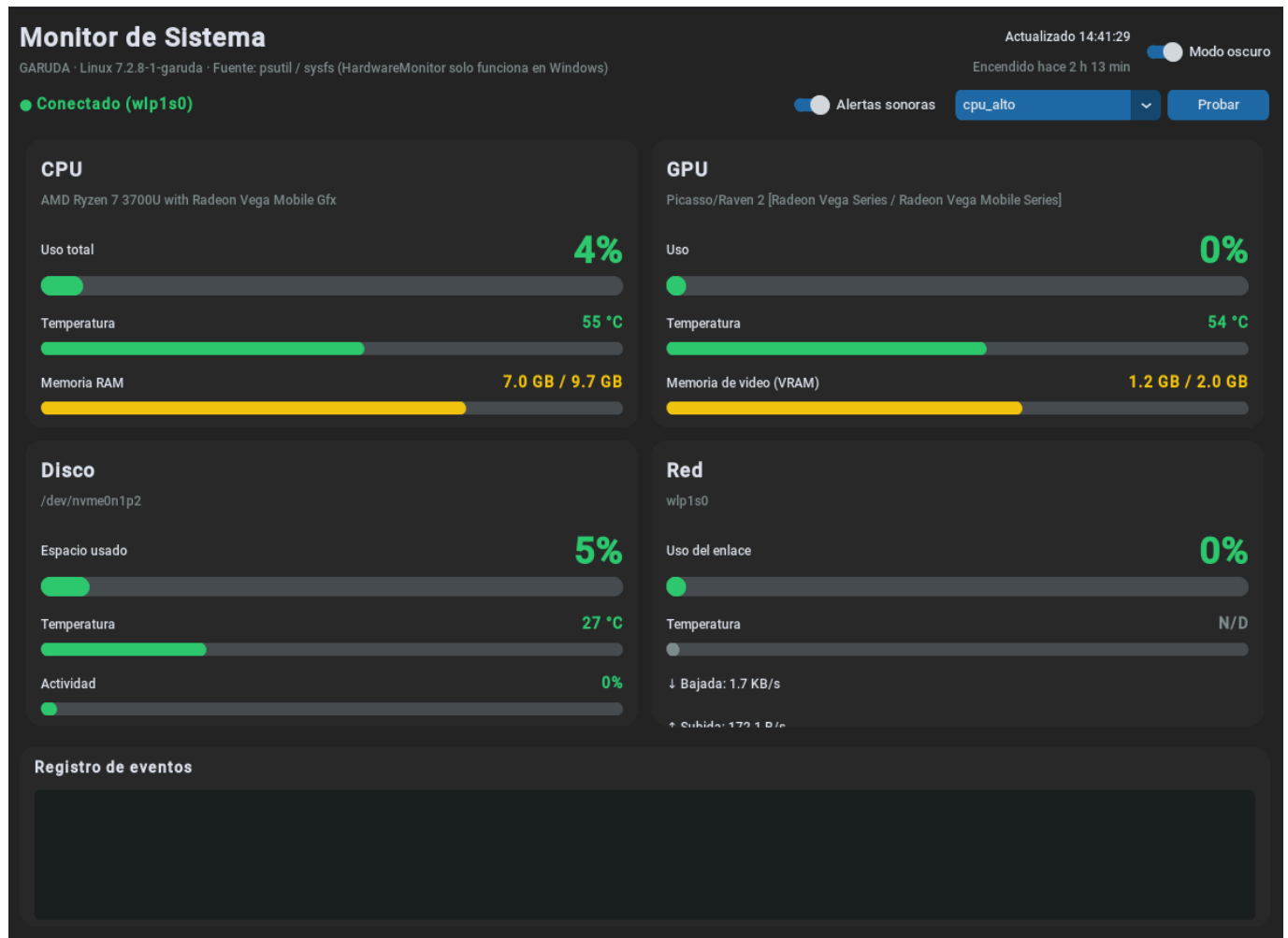
Alerta	Inicio (s)	Inicio de la anterior + duración + pausa (s)
cpu_alto	0.00	—
memoria_alta	0.75	0.00 + 0.33 + 0.4 = 0.73
red_trafico_alto	2.00	0.75 + 0.80 + 0.4 = 1.95
red_desconectada	3.01	2.00 + 0.42 + 0.4 = 2.82

Alerta	Inicio (s)	Inicio de la anterior + duración + pausa (s)
red_conectada	4.26	$3.01 + 0.75 + 0.4 = 4.16$
red_sigue_desconectada	5.26	$4.26 + 0.50 + 0.4 = 5.16$

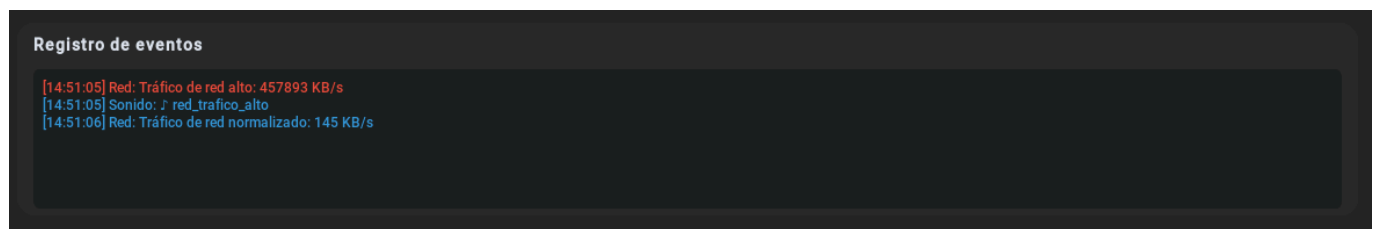
Además, la prueba unitaria `test_actualizar_no_espera_a_que_termine_el_sonido` verifica que `actualizar()` tarda mucho menos que la duración del sonido.

9. Capturas de pantalla de la ejecución

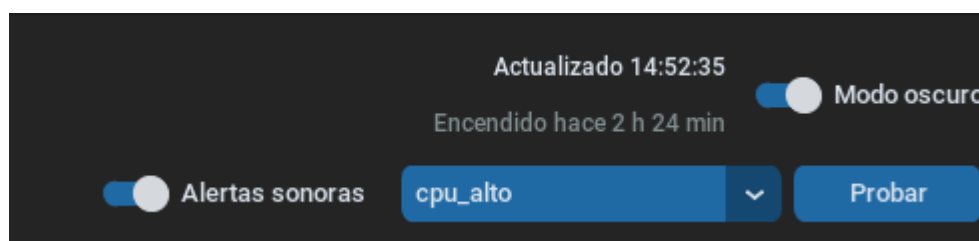
Figura del Dashboard en reposo:



Ejemplo de trafico Alto de Red:



Pruebas de Sonido y de alerta:



Prueba en vivo de las alertas (`python demo_alertas.py`), ejecutada con sonido real. Los latidos del ciclo continúan sin interrupción mientras suenan las alertas:

Backend: proceso externo (pw-play)

Alertas en cola: 6

```

    ♪ cpu_alto
[ciclo  1] t= 0.00 s  pendientes=5  sonando=sí
[ciclo  2] t= 0.25 s  pendientes=5  sonando=sí
[ciclo  3] t= 0.50 s  pendientes=5  sonando=sí
    ♪ memoria_alta
[ciclo  4] t= 0.75 s  pendientes=4  sonando=sí
[ciclo  5] t= 1.00 s  pendientes=4  sonando=sí
    ...
    ♪ red_trafico_alto
[ciclo  9] t= 2.00 s  pendientes=3  sonando=sí
    ...
    ♪ red_desconectada
[ciclo 13] t= 3.01 s  pendientes=2  sonando=sí
    ...
    ♪ red_conectada
[ciclo 18] t= 4.26 s  pendientes=1  sonando=sí
    ...
    ♪ red_sigue_desconectada
[ciclo 22] t= 5.26 s  pendientes=0  sonando=sí
    ...
[ciclo 25] t= 6.01 s  pendientes=0  sonando=sí

```

Ciclos ejecutados mientras sonaban las alertas: 25

Mayor duración de actualizar(): 1.92 ms

Modo consola del monitor (`python main.py --consola --silencioso`), con datos reales del equipo de desarrollo:

Fuente de datos: psutil / sysfs (HardwareMonitor solo funciona en Windows)
Sonido: proceso externo (pw-play) [SILENCIOSO]

--- 14:25:38 ---

CPU 0% 64 °C 8 hilos / 4 núcleos [AMD Ryzen 7 3700U with Radeon Vega Mobile Gfx]

RAM 72% 6.9 GB / 9.7 GB

DISCO 5% 29 °C 24.3 GB / 476.6 GB L 0.0 B/s E 0.0 B/s

GPU 0% 63 °C VRAM 1.1 GB / 2.0 GB [Picasso/Raven 2 [Radeon Vega Series / Radeon Vega Mobile Series]]

RED ↓ 0.0 B/s ↑ 0.0 B/s [wlp1s0]

10. Resultado de las pruebas unitarias

Se ejecutan con el módulo `unittest` de la biblioteca estándar:

```
python -m unittest -v
```

Resultado: **63 pruebas, todas correctas** (1.8 s).

Archivo	Pruebas	Qué verifica
<code>test_alertas.py</code>	27	Síntesis del WAV (formato, duración, silencio, volumen), sonidos distintos entre sí, cola <code>deque</code> (orden, sin solapamiento, pausa, duplicados, caducidad, tamaño máximo), modo silencioso, suscripción al bus, <code>actualizar()</code> sin bloqueo y backends
<code>test_conexion.py</code>	18	Flanco único de <code>red_desconectada</code> y <code>red_conectada</code> , recordatorio cada 30 s, rearme del ciclo, arranque sin red, sensor (IP válida, interfaz caída, loopback, enlace local, interfaces virtuales, IPv6, error del SO) e integración sensor → detector → bus → reproductor
<code>test_detectores.py</code>	12	Flanco, histéresis, rearme y umbrales inválidos de <code>DetectorUmbraI</code> ; consistencia de <code>config.DETECTORES</code> ; bus de eventos
<code>test_nodo.py</code>	6	Muestreo con sensores reales, tolerancia a un sensor que falla, arranque y parada del hilo, formatos y colores

```
test_crear_backend_usa_campana_si_no_hay_reproductor ... ok
test_crear_reproductor_queda_conectado_al_bus ... ok
test_el_backend_silencioso_no_hace_nada ... ok
test_el_proceso_externo_se_lanza_sin_esperar ... ok
test_la_campana_escribe_el_caracter_de_campana ... ok
test_actualizar_no_espera_a_que_termine_el_sonido ... ok
test_cerrar_cierra_el_backend ... ok
test_descarta_alertas_caducadas ... ok
test_emitir_el_aviso_no_provoca_nuevas_alertas ... ok
test_ignora_alertas_desconocidas ... ok
test_inicia_la_alerta_encolada ... ok
test_la_cola_tiene_tamano_maximo ... ok
test_la_pausa_entre_alertas_se_respeto ... ok
test_modo_silencioso_no_sueno_pero_consume_y_avisa ... ok
test_no_encola_duplicados_pendientes ... ok
test_no_solapa_alertas_y_respeto_el_orden ... ok
test_reacciona_a_los_eventos_del_bus ... ok
test_se_puede_reactivar_el_sonido ... ok
test_si_todas_caducaron_no_sueno_nada ... ok
test_sin_alertas_no_hace_nada ... ok
```

```
test_cada_sonido_es_distinto ... ok
test_el_catalogo_trae_wav_y_duracion ... ok
test_el_silencio_genera_muestras_en_cero ... ok
test_el_wav_es_valido_y_dura_lo_esperado ... ok
test_hay_sonido_para_cpu_memoria_trafico_y_conexion ... ok
test_los_sonidos_corresponden_a_eventos_con_mensaje ... ok
test_un_tono_no_supera_el_volumen_configurado ... ok
test_conectada_se_emite_una_sola_vez ... ok
test_conectado_desde_el_inicio_no_emite_nada ... ok
test_el_ciclo_se_rearma_tras_recuperarse ... ok
test_emite_conectada_al_recuperarse_con_el_tiempo_caído ... ok
test_emite_desconectada_una_sola_vez ... ok
test_los_nombres_de_eventos_son_los_del_enunciado ... ok
test_los_tres_eventos_tienen_mensaje_y_sonido ... ok
test_no_hay_recordatorio_si_la_red_esta_conectada ... ok
test_recordatorio_por_tiempo_mientras_siga_caída ... ok
test_si_arranca_sin_red_avisa_que_esta_desconectada ... ok
test_sin_dato_de_conexion_no_emite_ni_cambia_estado ... ok
test_la_caída_y_la_recuperacion_hacen_sonar_las_alertas_correctas ... ok
test_conectado_con_ip_valida ... ok
test_interfaces_virtuales_se_ignoran ... ok
test_interfaz_caída_no_cuenta ... ok
test_ipv6_global_cuenta ... ok
test_loopback_y_enlace_local_no_cuentan ... ok
test_un_error_del_sistema_se_interpreta_como_sin_conexion ... ok
test_entrega_a_suscriptor_del_evento_y_al_comodin ... ok
test_un_manejador_defectuoso_no_afecta_a_los_demas ... ok
test_cada_evento_tiene_mensaje ... ok
test_ids_y_eventos_son_unicos ... ok
test_todos_los_detectores_se_crean ... ok
test_el_evento_usa_la_marca_de_tiempo_de_la_instantanea ... ok
test_emite_alto_una_sola_vez ... ok
test_emite_normal_al_recuperarse_y_rearma ... ok
test_histeresis_no_emite_normal_en_zona_intermedia ... ok
test_no_emite_bajo_el_umbral ... ok
test_sin_dato_no_emite_ni_cambia_estado ... ok
test_umbrales_invertidos_son_rechazados ... ok
test_color_uso_por_niveles ... ok
test_formato_bytes ... ok
test_formato_velocidad ... ok
test_el_hilo_publica_instantaneas_y_se_detiene ... ok
test_muestrear_devuelve_cpu_memoria_y_disco ... ok
test_un_sensor_que_falla_no_rompe_el_muestreo ... ok
```

```
-----
Ran 63 tests in 1.829s
```

```
OK
```

Figura 11. Resultado de `python -m unittest -v`: 63 pruebas correctas.

Pruebas en vivo. Además de `demo_alertas.py` (sección 9), la caída real de la red puede probarse desactivando el WiFi o desconectando el cable con el dashboard abierto: debe aparecer «Sin conexión»,

sonar `red_desconectada`, repetirse `red_sigue_desconectada` cada 30 s y, al reconectar, sonar `red_conectada`.

11. Cumplimiento de los resultados esperados

Resultado del enunciado	Cumplimiento
E.1 Código con módulos y paquetes, con un nuevo paquete <code>alertas</code>	Paquete <code>alertas</code> / con <code>sonidos</code> , <code>backends</code> y <code>reproductor</code>
E.2 Eventos <code>red_desconectada</code> , <code>red_conectada</code> y <code>red_sigue_desconectada</code>	<code>DetectorConexion</code> (sección 6.2); nombres fijados en <code>config.CONEXION_EVENTOS</code>
E.3 Alertas distintas para CPU, memoria, tráfico y conexión, sin congelar el dashboard	<code>config.ALERTAS_SONORAS</code> (sección 7); prueba en vivo con 25 latidos ininterrumpidos (sección 8)
E.4 Pruebas unitarias con <code>unittest</code>	63 pruebas correctas (sección 10)
Consideración final	Cumplimiento
Marcar con <code># LABORATORIO</code> lo agregado o modificado	Sección 6.1
Sonidos, tiempos y umbrales solo en <code>config.py</code>	Sección 5.6
Modo silencioso para el salón	Sección 6.4
Declarar el uso de IA	Sección 13

12. Conclusiones

- La arquitectura sensores → detectores → bus de eventos permitió añadir el sonido como un suscriptor más. Los sensores y la lógica de `DetectorUmbral` quedaron intactos, y las únicas modificaciones en código existente fueron de conexión («cableado») en `nodo.py`, `main.py`, `consola.py` y `dashboard.py`.
- Los eventos por flanco con histéresis, junto con el recordatorio por tiempo, resuelven el problema del técnico: se entera una vez de que la red cayó, se le recuerda mientras siga caída y se le avisa cuando se recupera, sin repeticiones continuas.
- El sonido no bloquea porque nunca se espera a que termine: la emisión es asíncrona y el reproductor solo compara marcas de tiempo. La prueba en vivo mostró 25 latidos seguidos del ciclo mientras sonaban seis alertas, con `actualizar()` tardando como máximo 1.92 ms.
- Diferenciar los sonidos por contorno y ritmo, y no solo por tono, permite reconocer el tipo de problema de oído; las alertas de recordatorio se hicieron deliberadamente más discretas.
- Centralizar todos los valores en `config.py` permite ajustar sonidos, umbrales y tiempos sin tocar el código.
- **Limitaciones.** La conexión se determina localmente (interfaz activa con IP), no comprueba la salida a Internet. La ruta de `HardwareMonitor` para Windows no pudo probarse en el equipo de desarrollo (Linux). En Windows el reproductor usa `winsound`, también sin probar en el equipo de desarrollo.

13. Uso de herramientas de inteligencia artificial

El enunciado exige declarar el uso de IA, para qué partes y cómo se verificó el resultado. **Completar con lo que realmente aplique.** Borrador basado en este desarrollo:

Parte	Uso de IA	Verificación
Sistema base: módulos, sensores, detectores, bus, nodo y dashboard	Generación del código con un asistente de IA (Cursor)	Revisión manual, ejecución en modo consola con datos reales del equipo y pruebas unitarias
Mejora: <code>DetectorConexion</code> , <code>SensorConexion</code> y paquete <code>alertas</code>	Generación del código con el asistente	Revisión manual, 45 pruebas unitarias nuevas y prueba en vivo con sonido real (<code>demo_alertas.py</code>)
Diseño de los sonidos	Propuesta inicial del asistente	El estudiante debe escuchar los seis sonidos con <code>demo_alertas.py</code> y confirmar que se distinguen de oído (<i>completar</i>)
Informe	Redactado con el asistente	Revisión del estudiante
Rutas de Windows (HardwareMonitor y <code>winsound</code>)	Escritas con el asistente según la documentación de las APIs	No verificadas en el equipo de desarrollo; deben probarse en Windows

El estudiante debe poder explicar cualquier parte del código durante la demostración.

14. Anexo: instalación y ejecución

Linux (Arch / Garuda)

```
sudo pacman -S tk                # tkinter (necesario para
customtkinter)
cd ~/python-leo
python -m venv .venv              # esta distribución no incluye pip
global
source .venv/bin/activate
pip install -r requirements.txt
python main.py
```

Windows

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt    # incluye HardwareMonitor
(pythontk)
python main.py                    # ejecutar como administrador para
leer temperaturas
```

Opciones y pruebas

```
python main.py                    # interfaz gráfica
python main.py --silencioso      # sin sonido (modo salón)
python main.py --consola          # modo texto, sin interfaz gráfica
python demo_alertas.py            # prueba en vivo de las alertas
python -m unittest -v            # pruebas unitarias
```

Ajustes frecuentes (todo en `config.py`)

Qué ajustar	Constante
Frecuencia de muestreo	INTERVALO_MONITOREO_S
Umbral de alerta	DETECTORES
Sonidos de cada alerta	ALERTAS_SONORAS
Volumen	AUDIO_VOLUMEN
Pausa entre alertas, tamaño y caducidad de la cola	ALERTAS_PAUSA_ENTRE_S, ALERTAS_COLA_MAX, ALERTAS_CADUCIDAD_S
Intervalo del recordatorio de red	RED_RECORDATORIO_S

Qué ajustar

Constante

Modo silencioso por defecto

ALERTAS_SILENCIOSO_POR_DEFECTO